

Static Permissibility in Dynamic Normative Environments: Dynamic Classification Failure as a Structural Property of Institutional Computation

Ignacio Adrian Lerer

Independent Researcher | Buenos Aires, Argentina
adrian@lerer.com.ar | ORCID: 0009-0007-6378-9749

July 2026

Abstract

Computational systems deployed in consequence-bearing institutional contexts inherit an assumption that their governance frameworks rarely make explicit: that the normative conditions under which an action was classified as permissible remain stable until the action is executed. This paper argues that assumption is architecturally false and its failure is not incidental but structural. When an institution classifies a machine-generated action as permissible at time t_0 and relies on that classification at time t_1 without re-evaluation, it commits what this paper formalizes as Dynamic Classification Failure (DCF): reliance on a stale permissibility label in a normative environment that has changed between classification and consequence. DCF is not a governance oversight. It is a computational property that emerges from the mismatch between static authorization architectures and dynamic normative environments. This paper makes three contributions. First, it provides a formal definition of DCF as a state-transition failure in institutional computation, distinct from authorization defects, implementation errors, and design failures. Second, it situates DCF within the Generalized Intentionality Mismatch Theorem (GIMT; Lerer, 2025b), showing that systems operating at Level 0 or Level 1 of intentionality cannot detect changes in normative admissibility because their optimization architecture does not model institutional state as a variable. Third, it derives the necessary conditions for detecting and preventing DCF, connecting the formal analysis to existing legal and regulatory frameworks governing automated institutional action. The paper does not propose a reference implementation. It provides the theoretical

foundation for recognizing DCF as a failure category that current AI governance frameworks do not adequately address.

Keywords: Dynamic Classification Failure, institutional computation, normative environments, intentionality mismatch, static authorization, runtime admissibility, AI governance, agentic systems

1. Introduction

A financial institution authorizes an automated compliance system to screen transactions. At the time of authorization, the system receives a classification: transactions below a defined threshold, involving active accounts and unrestricted counterparties, are permissible. The system classifies a specific transaction as permissible at t_0 . Before execution at t_1 , the counterparty is added to a restricted list. The system executes the transaction. The institution has relied on a permissibility label that ceased to be accurate before the consequential action occurred.

This is not a hypothetical. Variants of this sequence occur wherever computational systems receive classifications at one point in time and act on them at another, within environments whose normative conditions change between those two moments. The failure pattern is not specific to financial compliance. It appears in automated legal case routing, algorithmic regulatory filings, AI-assisted procurement decisions, and agentic systems operating across changing legal jurisdictions.

Existing AI governance frameworks address the question of whether a system should be permitted to act, who is accountable for outputs, what documentation requirements apply, and how risk should be assessed in advance of deployment. These questions matter. They do not, however, address the computational properties required before an institution can legitimately treat a specific machine-generated action as having force in a dynamic normative environment. That gap is the subject of this paper.

Section 2 introduces the state-transition model of institutional computation and defines Dynamic Classification Failure formally within that model. Section 3 situates DCF within the Generalized Intentionality Mismatch Theorem and explains why Level 0 and Level 1 systems cannot self-detect DCF. Section 4 derives the necessary conditions for DCF prevention and relates them to existing legal and regulatory frameworks. Section 5 examines three DCF failure modes and their institutional implications. Section 6 discusses the relationship between DCF and runtime admissibility as a necessary computational primitive for institutional reliance. Section 7 addresses limitations. Section 8 concludes.

2. Formal Definition of Dynamic Classification Failure

2.1 Institutional State as a Variable

An institution I at any time t can be described by an institutional state St , which encodes the set of legal duties, evidentiary conditions, policy constraints, delegated authority, procedural status, and consequence thresholds that are operative at t . Institutional state is not static. It changes as a function of regulatory modifications, judicial decisions, counterparty status changes, revocation of delegated authority, introduction of new evidentiary material, and transitions in procedural posture.

Let St be the institutional state at time t , and let x be a proposed machine-generated action. A permissibility function $\text{Perm}(x, St)$ returns true if action x is permissible under institutional state St , and false otherwise.

2.2 The Static Agent Assumption

A computational system that classifies actions without modeling institutional state as a variable implicitly adopts what I term the Static Agent Assumption: the assertion that the normative conditions operative at classification time t_0 will remain operative at execution time t_1 . Under this assumption, if $\text{Perm}(x, St_0) = \text{true}$, then $\text{Perm}(x, St_1) = \text{true}$ for all $t_1 > t_0$.

The Static Agent Assumption is incoherent in any normative environment where institutional state changes between classification and execution. It is not merely an empirical error about whether states do in fact change. It is an architectural commitment to ignoring state change as a variable, regardless of whether change occurs.

2.3 Dynamic Classification Failure Defined

Dynamic Classification Failure occurs when all four of the following conditions hold:

- (i) A system classifies action x as permissible under state St_0 : $\text{Perm}(x, St_0) = \text{true}$.
- (ii) The institutional state changes between classification and execution: $St_0 \neq St_1$.
- (iii) The institution relies on action x at time t_1 based on the classification obtained at t_0 : $\text{Rel}(x, t_1)$.
- (iv) The system does not re-evaluate x under current state St_1 before relying on it: $\neg \text{Reeval}(x, St_1)$.

Formally: $\text{DCF}(x, t_0, t_1)$ iff $\text{Perm}(x, St_0) = \text{true} \wedge St_0 \neq St_1 \wedge \text{Rel}(x, t_1) \wedge \neg \text{Reeval}(x, St_1)$.

When the state change is such that $\text{Perm}(x, St_1) = \text{false}$, DCF produces defective reliance: the institution has relied on an action that is no longer permissible under current conditions. The institution has acted, or allowed an action to proceed, on the basis of a classification

that does not accurately represent the normative environment at the moment of consequence.

2.4 Distinguishing DCF from Adjacent Failure Modes

DCF is distinct from three failure modes with which it may be confused. The distinctions matter because each failure mode is addressed by a different governance instrument, and the instrument adequate to each of the three is inadequate to DCF.

Authorization failure occurs when a system acts outside its delegated authority at t_0 . DCF presupposes the opposite: authorization was valid at t_0 and the classification was accurate at t_0 . The defect is not in the grant of authority but in the interval between authorization and reliance. An authorization audit conducted at t_0 produces a clean record. The defective reliance occurs at t_1 , after the audit's temporal scope has closed.

Implementation error occurs when a system correctly classifies an action but executes it incorrectly. DCF concerns the classification itself, not its technical execution. Implementation testing verifies that system outputs match their specified inputs. It does not verify that the normative conditions underlying those inputs remain operative at the moment of reliance.

Design failure occurs when a system is built with objectives or constraints that produce harmful outputs under any conditions. DCF is a runtime property, not a design property. A system with no design defect whatsoever can produce DCF: the same architecture that classifies correctly at t_0 generates defective reliance at t_1 because the normative environment changed, not because the system was built wrongly. Design-time evaluation therefore cannot detect it.

The three governance instruments that correspond to these failure modes, authorization audits, implementation testing, and design-time evaluation, each operate before or at t_0 . None of them reaches the interval between t_0 and t_1 where DCF occurs. DCF requires a distinct instrument: a mechanism that evaluates normative admissibility at the moment of institutional reliance, not only at authorization time or deployment time.

3. DCF and the Generalized Intentionality Mismatch Theorem

3.1 The GIMT and Intentionality Levels

The Generalized Intentionality Mismatch Theorem (GIMT; Lerer, 2025b) formalizes the observation that legal and governance regimes embody implicit cognitive assumptions about the agents they regulate. Major AI governance frameworks, including the NIST AI Risk Management Framework (NIST, 2023) and the EU AI Act (European Union, 2024), assume that regulated systems occupy a stable risk classification that governance controls can be calibrated to. That assumption is the institutional version of the Static Agent

Assumption identified in Section 2.2. Dennett's (1987) intentionality hierarchy provides the conceptual scaffolding for understanding why it fails:

- Level 0: Systems responding to physical stimuli via fixed rules, without goal representation.
- Level 1: Goal-directed optimizers that adjust behavior in response to environmental constraints but cannot model others' mental states.
- Level 2: Systems that model others' beliefs.
- Level 3: Agents capable of recursive belief modeling, social emotion, and norm internalization.

The GIMT identifies six failure modes that emerge when governance regimes assume a level of intentionality that the regulated system does not in fact possess. DCF is the sixth failure mode.

3.2 DCF as the Sixth GIMT Mode

The original five GIMT failure modes concern static population structures: some actors are persistently at one intentionality level, and the governance regime is persistently calibrated to a different level. DCF is structurally distinct. It does not require a population of actors at different levels. It requires only that a computational system, regardless of its intentionality level, relies on a classification obtained under conditions that no longer obtain.

DCF is distinct from the other five modes in a specific sense: even a system that has been correctly authorized under a valid governance regime, and that operates without any intentionality mismatch at the moment of classification, can produce DCF if it is architecturally incapable of detecting that the normative environment changed between classification and execution.

This architectural incapacity is not a property of low-capability systems alone. A highly capable system that models institutional states but does not continuously monitor them for changes between classification and execution is vulnerable to DCF. The relevant architectural property is not the system's general capability but whether it incorporates normative state as a live variable at the moment of action, rather than as a fixed input at the moment of classification.

3.3 Why Level 0 and Level 1 Systems Cannot Self-Detect DCF

A Level 0 system operates via fixed rules. It has no mechanism for monitoring environmental changes and no representation of institutional state as a variable that could become stale. DCF is not detectable at Level 0.

A Level 1 system is a goal-directed optimizer. It adjusts behavior in response to observable constraints, but it does not model the normative properties of those constraints. A Level 1 system treats a permissibility label as a constraint parameter in its optimization problem. If the label is not updated, the system continues to optimize against the original label. The system has no architecture for detecting that the label no longer accurately describes the normative environment.

This is not a failure of sophistication in the ordinary sense. A Level 1 system can be extremely capable within its optimization architecture. The point is that the optimization architecture itself does not include a self-monitoring function over the continued validity of its normative inputs. That function requires architectural support external to the optimization loop.

The implication is that DCF prevention cannot be achieved by improving the design-time accuracy of permissibility classifications. The classification may be perfectly accurate at t_0 and still produce DCF at t_1 if the normative environment changes and the system has no mechanism for detecting that change.

4. Necessary Conditions for DCF Prevention

4.1 State Monitoring as a Structural Requirement

DCF prevention requires that the institutional state operative at the moment of reliance, St_1 , be evaluated against the permissibility of the proposed action before the action is treated as admissible. This is a structural requirement, not a governance recommendation. Without it, any system deployed in a dynamic normative environment can produce DCF, regardless of the quality of its design-time authorization.

State monitoring is distinct from state logging. Logging records states for audit purposes after the fact; it cannot detect DCF before it occurs. State monitoring means that the current state is an active input to the admissibility evaluation at t_1 , not merely an archived record of what was true at t_0 .

4.2 Re-evaluation Triggers

Not every interval between classification and execution requires continuous re-evaluation. DCF prevention requires that re-evaluation occurs when any of the following conditions holds:

- (a) Authority conditions: the delegated authority under which the classification was made has changed, expired, been revoked, or been narrowed.
- (b) Counterparty or subject status: the status of any party or entity material to the permissibility determination has changed.

- (c) Evidentiary basis: material evidence supporting the permissibility classification has been contradicted, updated, or found insufficient.
- (d) Consequence threshold: the magnitude or class of consequence to be produced by the action has changed since the classification was obtained.
- (e) Regulatory or legal conditions: any legal rule, regulation, or policy material to the permissibility determination has changed.

The practical implementation of these triggers varies by domain and system architecture. The structural requirement is that the system have a mechanism for detecting that any of these conditions has changed and for routing the action to re-evaluation when they have.

4.3 Relationship to Existing Legal and Regulatory Frameworks

Several existing legal and regulatory frameworks are implicitly sensitive to DCF, though they do not use that terminology.

Article 22 of Regulation (EU) 2016/679 (GDPR) restricts automated individual decision-making that produces legal or other significant effects, and requires that such decisions be subject to human review. The right to review functions, in part, as a mechanism for detecting DCF: the human reviewer can identify cases where the normative conditions operative at classification time have changed before the decision takes effect.

Articles 9 and 29 of Regulation (EU) 2024/1689 (EU AI Act) impose risk management and monitoring obligations on providers and deployers of high-risk AI systems. Monitoring obligations extend to post-deployment operation, creating at least a partial basis for requiring state re-evaluation in systems whose operating conditions change.

Articles 1757 and 1758 of the Argentine Civil and Commercial Code (Codigo Civil y Comercial de la Nacion, Law 26.994) establish objective liability for risk-creating activities, applicable to the owner and custodian of a thing or activity that creates risk. Applied to AI systems, these provisions create a liability structure in which an institution that relies on a machine-generated action under changed normative conditions may bear liability for resulting harm, regardless of whether the original classification was accurate.

These frameworks create regulatory pressure for DCF prevention without formally specifying the computational properties required. The formal definition in Section 2 provides the missing specification.

5. Three DCF Failure Modes and Their Institutional Implications

5.1 Authority Drift

Authority Drift occurs when the delegated authority under which an action was classified as permissible at t_0 is revoked, expired, or narrowed before execution at t_1 , and the system executes the action without detecting the change in authority.

The institutional implication of Authority Drift is that the institution may have produced a consequential action without valid delegated authority. In administrative law terms, this is analogous to the *ultra vires* doctrine: the actor has exceeded the scope of valid authority, even if the original authorization was valid. Authority Drift is not *ultra vires* from the beginning; it becomes so between t_0 and t_1 .

5.2 Evidence Collapse

Evidence Collapse occurs when the evidentiary basis for a permissibility classification becomes contradicted, outdated, or insufficient before execution, and the system executes the action without re-evaluating the classification against the current evidentiary basis.

Evidence Collapse has particular relevance in contexts where permissibility determinations depend on facts about third parties, because third-party facts can change independently of the system's operations and may not be automatically reported to the system. A counterparty's financial status, regulatory standing, or contractual obligations can change between the time of classification and the time of execution, invalidating an evidentiary basis that was accurate when the classification was made.

5.3 Normative Phase Shift

Normative Phase Shift occurs when the legal or regulatory rules material to a permissibility determination change between t_0 and t_1 , and the system executes the action under the original normative framework without detecting the change.

Normative Phase Shift is more tractable than the other two failure modes in some respects: legal and regulatory changes often have known effective dates, allowing systems to be designed to trigger re-evaluation on those dates. It is less tractable in other respects: judicial decisions that change the interpretation of existing rules do not always have announced effective dates, and their implications for specific permissibility classifications may not be immediately apparent.

6. DCF and Runtime Admissibility

The preceding analysis establishes DCF as a structural failure category. It does not, by itself, specify what computational properties are required to prevent it. The concept of runtime admissibility (Miteiko and Lerer, 2026) provides that specification.

Runtime admissibility is the computational determination that a proposed machine-generated action remains institutionally acceptable at the moment of reliance, given current

authority, evidence, institutional state, consequence class, execution integrity, and correction conditions. The runtime admissibility predicate evaluates these conditions at t_1 , not at t_0 . It therefore provides a direct computational mechanism for DCF detection.

The relationship between DCF and runtime admissibility can be stated precisely: DCF is a necessary condition for defective institutional reliance. Runtime admissibility operationalizes the evaluation required to detect whether that condition holds at the moment of consequence. A system that evaluates runtime admissibility before acting cannot produce DCF, because it re-evaluates normative permissibility at t_1 rather than relying on the classification obtained at t_0 .

This paper has established why DCF is a structural failure category, why it cannot be prevented by design-time authorization alone, and why it cannot be self-detected by Level 0 or Level 1 systems. The companion paper (Miteiko and Lerer, 2026) specifies the computational primitive required to operationalize DCF prevention.

7. Limitations

This paper establishes DCF as a theoretical failure category and derives necessary conditions for its prevention. It does not:

- (a) Propose a reference implementation or software architecture for DCF prevention systems. Implementation-specific architectures depend on domain, system type, and institutional context and are treated as separate work.
- (b) Establish sufficient conditions for legitimate institutional reliance. Runtime admissibility addresses DCF but does not address all conditions required for a machine-generated action to have legal force. Jurisdiction-specific requirements, professional duties, procedural rules, and ethical constraints impose additional conditions.
- (c) Provide an empirical survey of DCF incidence. Such a survey would require access to production systems in institutional contexts, which this paper does not have.
- (d) Address DCF in multi-institutional settings where authority envelopes and evidentiary bases are distributed across institutions with different classification timelines. Cross-institutional DCF is a tractable extension but is outside the scope of this paper.

The formal model in Section 2 is intentionally minimal. It captures the structure required to distinguish DCF from adjacent failure categories. Extensions using richer temporal logic, probabilistic evidentiary models, or multi-agent delegation structures are directions for future work.

8. Conclusion

Dynamic Classification Failure is the structural failure that occurs when a computational system relies on a permissibility classification obtained under institutional conditions that have changed before the action producing institutional consequence is executed. It is not an authorization failure, an implementation error, or a design defect. It is a runtime property of computational systems operating in dynamic normative environments.

DCF cannot be prevented by improving design-time authorization, however thorough that authorization may be. It cannot be self-detected by systems that treat permissibility as a fixed input rather than as a function of current institutional state. It requires architectural support: a mechanism that evaluates normative admissibility at the moment of institutional reliance, with access to current authority, evidence, and regulatory conditions.

Existing AI governance frameworks create regulatory pressure for DCF prevention without specifying the computational properties required. This paper provides that specification. The result is a failure category that can be named, defined, distinguished from adjacent failures, and operationalized through the concept of runtime admissibility as a computational primitive.

Where institutions rely on machine-generated actions to create institutional consequences, they must be able to determine whether the normative conditions that made those actions permissible remain operative at the moment of reliance. The alternative is not merely regulatory non-compliance. It is computationally defective reliance: acting on a classification of the world that no longer describes the world.

References

- Dennett, D. C. (1987). *The Intentional Stance*. MIT Press.
- European Union (2016). Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data (GDPR). *Official Journal of the European Union*, L 119, 1-88.
- European Union (2024). Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). *Official Journal of the European Union*, L 2024/1689.
- Lakatos, I. (1978). *The Methodology of Scientific Research Programmes*. Cambridge University Press.
- Lerer, I. A. (2025a). Asymmetric Intentionality in Legal Games: When Players Operate at Different Intentional Levels. Zenodo. DOI: 10.5281/zenodo.18603169.

- Lerer, I. A. (2025b). The Generalized Intentionality Mismatch Theorem: Failure Modes of Human-Institutional Interaction. Zenodo. DOI: 10.5281/zenodo.18857385.
- Miteiko, A. and Lerer, I. A. (2026). Runtime Admissibility as a Computational Primitive for Institutional Reliance. Manuscript under review.
- National Institute of Standards and Technology (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1. U.S. Department of Commerce.
- Popper, K. R. (1959). *The Logic of Scientific Discovery*. Routledge.
- Republica Argentina (2014).Codigo Civil y Comercial de la Nacion (Ley 26.994). Arts. 1757-1758.

Lakatosian Epistemological Declaration

The theoretical claims in this paper are falsifiable in the sense of Popper (1959) and Lakatos (1978). The core claim, that Dynamic Classification Failure is a structural property of computational systems operating in dynamic normative environments, is falsifiable by demonstrating a class of systems that: (a) operate without continuous state monitoring; (b) are deployed in normative environments whose conditions change between classification and execution; and (c) do not produce DCF. The necessary conditions derived in Section 4 generate testable predictions about which systems are DCF-vulnerable. These predictions can be evaluated against controlled experiments of the type proposed by Miteiko and Lerer (2026) and against case studies of institutional failure attributable to stale permissibility classifications.

AI Disclosure: This paper was drafted with assistance from Claude (Anthropic) as a research and writing tool. The theoretical framework, formal definitions, and arguments are the author's original contribution. All citations were verified against primary sources. The AI assistant's outputs were evaluated critically and revised substantially prior to finalization.